MACHINE LEARNING

PRACTICAL RECORD

Aim • Objectives • Theory • Algorithm

Total Experiments	15
Programming Language	Python
Primary Libraries	NumPy, Pandas, Matplotlib, scikit-learn
Topics	Regression, Classification, Clustering, PCA, Model Tuning

Prepared as a theory and algorithm reference for laboratory practicals. Dataset-specific code, output, and observations can be added during execution.

General Notes

1. Use the dataset prescribed by your instructor. If no dataset is prescribed, select a suitable sample dataset and clearly mention its source.
2. Set a random_state for reproducible train-test splits and stochastic estimators wherever supported.
3. Fit preprocessing steps only on training data. Use scikit-learn Pipeline and ColumnTransformer where suitable to prevent data leakage.
4. Do not use the test set to repeatedly select hyperparameters. Use validation data or cross-validation for model selection, then report final test performance.
5. Record the code, dataset details, key outputs, plots, evaluation metrics, and a short conclusion separately if your practical journal requires them.

Experiment 1

Study and Implementation of Python Libraries for Machine Learning

Aim

To study and use NumPy, Pandas, and Matplotlib for numerical computation, data handling, analysis, and visualization.

Objectives

1. Understand arrays, indexing, slicing, reshaping, and vectorized operations using NumPy.
2. Load, inspect, clean, filter, and summarize tabular data using Pandas.
3. Create suitable plots using Matplotlib to understand distributions, trends, and relationships.
4. Combine the libraries in a small data-analysis workflow.

Theory

NumPy is a Python library for efficient numerical computing. Its main structure is the homogeneous, multidimensional ndarray. It supports vectorized arithmetic, broadcasting, aggregation, reshaping, and linear algebra operations. These operations are generally more concise and efficient than manually iterating through Python lists.

Pandas provides Series and DataFrame structures for labelled one-dimensional and tabular data. It supports reading files such as CSV, inspecting data types, selecting rows and columns, handling missing values, removing duplicates, grouping, merging, and computing descriptive statistics.

Matplotlib is a plotting library used to communicate patterns in data. Common plots include line charts for trends, bar charts for category comparisons, histograms for distributions, and scatter plots for relationships between numerical variables. Plot labels, legends, titles, and scales improve interpretation.

In a typical analysis, Pandas is used to load and prepare the data, NumPy supports numerical transformations, and Matplotlib visualizes the results.

Algorithm

1. Import NumPy, Pandas, and Matplotlib's pyplot module.
2. Create or load a sample dataset into a Pandas DataFrame.
3. Display the first few rows, dimensions, column names, data types, and summary statistics.
4. Check for missing values and duplicate records; clean them where appropriate.
5. Select relevant columns and perform basic filtering, aggregation, and numerical calculations.

6. Convert selected data to NumPy arrays and demonstrate array operations such as indexing, slicing, and mean calculation.
7. Plot the data using suitable charts such as a histogram, bar chart, line chart, or scatter plot.
8. Add titles, axis labels, and legends; display the plots and interpret the observations.

Experiment 2

Complete Machine Learning Workflow Using scikit-learn

Aim

To implement the complete machine learning workflow, from data loading and preprocessing to model training, testing, and evaluation, using scikit-learn.

Objectives

1. Load a dataset and identify its features and target variable.
2. Perform data cleaning, encoding, scaling, and train-test splitting as required.
3. Train a suitable supervised learning model using scikit-learn.
4. Generate predictions and evaluate model performance using appropriate metrics.

Theory

A machine learning workflow is a sequence of steps that converts raw data into a model that can make predictions on unseen examples. The first step is understanding the dataset, including its shape, data types, target, missing values, and class distribution.

Preprocessing prepares data for learning. Typical operations include imputing missing values, encoding categorical variables, scaling numerical features, and separating features (X) from the target (y). The data is split into training and testing subsets so that final performance is measured on data not used to fit the model.

scikit-learn provides consistent APIs: fit() learns from training data, predict() generates outputs, and score() or metric functions evaluate performance. Classification metrics include accuracy, precision, recall, F1-score, and a confusion matrix. Regression metrics include MAE, MSE, RMSE, and R-squared.

Preprocessing must be learned only from the training data. A Pipeline helps prevent data leakage by fitting transformations within each training fold and then applying them to validation or test data.

Algorithm

1. Import the required Pandas, scikit-learn preprocessing, model-selection, estimator, and metric modules.
2. Load the dataset and inspect its dimensions, columns, data types, missing values, and target distribution.
3. Separate the input features X from the target y; remove identifiers or irrelevant columns if justified.
4. Identify numerical and categorical columns. Impute missing values, encode categories, and scale numerical features when needed.
5. Split the data into training and testing sets using train_test_split; use stratification for classification when appropriate.
6. Create a preprocessing-and-model Pipeline to avoid data leakage.
7. Select a suitable estimator and train it using fit(X_train, y_train).
8. Predict the test labels or values using predict(X_test).
9. Evaluate with metrics suitable for the task and inspect errors or a confusion matrix where applicable.
10. Record the results and state limitations; do not tune the model on the final test set.

Experiment 3

Bias, Variance, Overfitting, and Underfitting Using Polynomial Regression

Aim

To analyze bias, variance, overfitting, and underfitting by fitting polynomial regression models of different degrees to a dataset.

Objectives

1. Understand the bias-variance trade-off in supervised learning.
2. Transform input features into polynomial features of different degrees.
3. Compare training and testing errors for models of increasing complexity.
4. Identify underfitting and overfitting from model performance and fitted curves.

Theory

Polynomial regression models a nonlinear relationship by creating powers of an input variable. A degree-d model has the form y = b0 + b1x + b2x^2 + ... + bd x^d. Although the model is nonlinear in x, it is linear in its coefficients and can be fitted using linear regression.

Bias is error caused by overly simple assumptions. A high-bias model may fail to capture the underlying pattern and underfit both training and test data. Variance describes sensitivity to fluctuations in the training data. A high-variance model may fit noise and perform well on training data but poorly on unseen data.

Underfitting is commonly indicated by high training and test errors. Overfitting is indicated by very low training error but substantially higher test error. A suitable model balances the two and generalizes well. A validation set or cross-validation can help select polynomial degree.

Feature scaling and a careful train-test split are important, especially for high-degree polynomial terms, which can become numerically large.

Algorithm

1. Load the dataset and identify the input variable X and continuous target y.
2. Split the data into training and testing sets before fitting transformations.
3. For each selected polynomial degree (for example, 1, 2, 3, 5, and 10), create polynomial features using PolynomialFeatures.
4. Fit a LinearRegression estimator on the transformed training data.
5. Predict the target for both training and testing data.
6. Calculate training and testing errors, such as mean squared error or RMSE, for each degree.
7. Plot the observed data and fitted curves for the different polynomial degrees.
8. Compare the errors and curves: high errors on both sets suggest underfitting; a large train-test gap suggests overfitting.
9. Choose a degree that gives a reasonable validation/test performance and explain the bias-variance trade-off.

Experiment 4

Simple Linear Regression

Aim

To implement Simple Linear Regression for predicting a continuous dependent variable from one independent variable and evaluate its performance.

Objectives

1. Understand the relationship between one predictor and a continuous target.
2. Fit a straight-line regression model using scikit-learn.
3. Interpret the slope and intercept of the fitted line.
4. Evaluate predictions using suitable regression error metrics.

Theory

Simple Linear Regression models the relationship between one independent variable x and a continuous dependent variable y. Its equation is y-hat = b0 + b1x, where b0 is the intercept and b1 is the slope. The slope estimates the expected change in y for a one-unit increase in x.

The ordinary least squares method estimates coefficients by minimizing the sum of squared differences between actual and predicted target values. The model is suitable when a roughly linear relationship is reasonable and observations meet the assumptions relevant to the analysis.

Mean Absolute Error (MAE) is the average absolute prediction error. Mean Squared Error (MSE) is the average squared error and penalizes larger errors more strongly. Root Mean Squared Error (RMSE) is the square root of MSE and is expressed in the target's units. R-squared measures the proportion of target variance explained by the model relative to a baseline mean predictor.

Algorithm

1. Import Pandas, train_test_split, LinearRegression, and regression metrics.
2. Load the dataset and select one independent feature X and one continuous target y.
3. Inspect missing values and plot X against y to examine whether a linear relationship is plausible.
4. Split the dataset into training and testing subsets.
5. Create a LinearRegression model and fit it using the training data.
6. Predict target values for the test data.
7. Display the model intercept and coefficient (slope).
8. Calculate MAE, MSE, RMSE, and R-squared on the test set.
9. Plot the data points and fitted regression line, then interpret the metrics and model limitations.

Experiment 5

Multiple Linear Regression

Aim

To implement Multiple Linear Regression using two or more independent features and evaluate it using MAE, MSE, RMSE, and R-squared.

Objectives

1. Model a continuous target using multiple explanatory variables.
2. Prepare numerical and categorical features for regression.
3. Fit and interpret a multiple linear regression model.

4. Evaluate predictive performance using four standard regression metrics.

Theory

Multiple Linear Regression extends simple linear regression to several predictors. Its equation is y-hat = b0 + b1x1 + b2x2 + ... + bkxk. Each coefficient represents the estimated change in the target for a one-unit change in that feature, holding the other included features constant.

The coefficients are commonly estimated using ordinary least squares, which minimizes the sum of squared residuals. Strong correlation among predictors (multicollinearity), outliers, omitted variables, and nonlinearity can affect coefficient stability or model adequacy.

MAE measures average absolute error; MSE measures average squared error; RMSE is the square root of MSE and is in the target's units; R-squared measures variance explained relative to a mean-only baseline. A high R-squared does not by itself establish causation or guarantee good predictions on new data.

Algorithm

1. Import the required Pandas and scikit-learn modules.
2. Load the dataset and select multiple independent variables as X and the continuous target as y.
3. Inspect the data, handle missing values, and encode categorical features if present.
4. Split the data into training and testing sets.
5. Fit a LinearRegression model using X_train and y_train.
6. Predict values for X_test.
7. Display the intercept and coefficients, matching each coefficient to its feature.
8. Calculate MAE, MSE, RMSE, and R-squared using the test predictions.
9. Compare actual and predicted values and interpret model performance.

Experiment 6

Regularization: Ridge, Lasso, and Elastic Net Regression

Aim

To implement and compare Ridge, Lasso, and Elastic Net regression and study how regularization improves model generalization.

Objectives

1. Understand the need for regularization in regression.
2. Compare L2, L1, and combined L1-L2 penalties.
3. Fit the three models and examine coefficient shrinkage.
4. Select regularization strength using validation or cross-validation.

Theory

Regularization adds a penalty to the regression loss to discourage unnecessarily large coefficients. It can reduce variance and help control overfitting, particularly when there are many features or correlated predictors. Numerical features should generally be scaled before applying penalized regression.

Ridge regression uses an L2 penalty, proportional to the sum of squared coefficients. It shrinks coefficients toward zero but usually does not make them exactly zero. Lasso uses an L1 penalty, proportional to the sum of absolute coefficients, and can set some coefficients exactly to zero, thereby performing feature selection.

Elastic Net combines L1 and L2 penalties. It can retain the feature-selection behavior of Lasso while handling groups of correlated features more stably than pure Lasso in some settings. The regularization strength controls the trade-off between fitting the data and keeping coefficients small.

Model comparison should use the same data split or cross-validation folds and a metric appropriate to the regression task. Scaling and hyperparameter selection must be performed within the training process to avoid leakage.

Algorithm

1. Load the dataset and separate the feature matrix X and continuous target y.
2. Split the data into training and testing sets.
3. Build a Pipeline that scales numerical features and applies the regression estimator.
4. Create Ridge, Lasso, and ElasticNet models with initial regularization parameters.
5. Fit each model on the training data and predict the test target.
6. Calculate comparable test metrics such as MAE, RMSE, and R-squared.
7. Record and compare the coefficients; note whether Lasso or Elastic Net sets any coefficients to zero.
8. Optionally use GridSearchCV or cross-validation on the training set to select alpha and Elastic Net l1_ratio.
9. Compare the tuned models and identify which gives the best validation performance and reasonable complexity.

Experiment 7

Logistic Regression for Binary Classification

Aim

To implement Logistic Regression for binary classification and evaluate its performance using a confusion matrix and accuracy score.

Objectives

1. Understand binary classification and probability estimation.
2. Train a Logistic Regression classifier using scikit-learn.
3. Convert predicted probabilities into class labels using a decision threshold.
4. Evaluate classification results using a confusion matrix and accuracy.

Theory

Despite its name, Logistic Regression is a classification algorithm. It computes a linear score from input features and applies the logistic (sigmoid) function to estimate the probability of the positive class: p = 1 / (1 + exp(-z)), where z = b0 + b1x1 + ... + bkxk.

A default threshold of 0.5 commonly assigns a positive label when the estimated probability is at least 0.5. The threshold can be changed according to the cost of false positives and false negatives. Feature scaling can help optimization, and class imbalance may make accuracy misleading.

A confusion matrix compares actual and predicted classes. For binary classification it contains true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN). Accuracy is (TP + TN) / (TP + TN + FP + FN). Precision, recall, and F1-score provide additional information, especially for imbalanced datasets.

Algorithm

1. Import the dataset, train_test_split, preprocessing tools, LogisticRegression, and classification metrics.
2. Separate the input features X and binary target y; inspect class balance and missing values.
3. Preprocess the features, including imputation, encoding, and scaling where appropriate.

4. Split the data into training and testing sets, using stratification when appropriate.
5. Create and fit a LogisticRegression classifier on the training data.
6. Predict class labels for the test set; optionally inspect predict_proba outputs.
7. Generate the confusion matrix and calculate accuracy, precision, recall, and F1-score.
8. Interpret false positives and false negatives and discuss whether the default threshold is suitable.

Experiment 8

K-Nearest Neighbours (KNN) Classification

Aim

To implement the K-Nearest Neighbours classification algorithm and analyze how different values of K affect model accuracy.

Objectives

1. Understand instance-based, distance-based classification.
2. Train KNN classifiers with different neighbour counts.
3. Study the effect of feature scaling and the choice of distance metric.
4. Compare training and test accuracy to identify a suitable K.

Theory

K-Nearest Neighbours is a supervised, non-parametric algorithm. To classify a new sample, it calculates its distance from training samples, selects the K closest neighbours, and assigns the majority class among them. Common distance measures include Euclidean and Manhattan distance.

A very small K can produce a sensitive, high-variance model that responds to noise. A very large K can oversmooth decision boundaries and underfit. The best K depends on the dataset and should be selected using validation or cross-validation rather than the test set.

Because distances depend on feature scales, standardization or normalization is usually important when features have different units or ranges. KNN can be computationally expensive at prediction time because it must compare a query with stored training samples.

Algorithm

1. Load the classification dataset and separate X and y.
2. Split the data into training and testing subsets, using stratification where suitable.
3. Create a Pipeline that scales the features and applies KNeighborsClassifier.
4. Choose a range of odd K values, for example 1, 3, 5, 7, 9, and 11, when binary-class ties are a concern.
5. For each K, fit the model on the training data and predict the test labels.
6. Calculate and record training and test accuracy for each K.
7. Plot K against test accuracy and inspect the train-test gap.
8. Select K using validation or cross-validation, then report final test performance once.

Experiment 9

Decision Tree Classification and Effect of Tree Depth

Aim

To implement a Decision Tree Classifier and study the effect of maximum tree depth on classification performance.

Objectives

1. Understand tree-based classification using feature-based splits.
2. Train decision trees with different maximum depths.
3. Evaluate how model complexity affects training and testing performance.
4. Identify signs of overfitting and underfitting.

Theory

A Decision Tree is a supervised learning model that recursively partitions the feature space. At each node, it selects a feature and split that improves class purity. Common split criteria include Gini impurity and entropy (information gain). A leaf node assigns a class or class probabilities.

Tree depth controls model complexity. A shallow tree may be unable to capture important patterns and underfit. A very deep tree can memorize training examples and overfit. Other controls include minimum samples per split, minimum samples per leaf, and cost-complexity pruning.

Decision trees are easy to interpret and do not generally require feature scaling. Their predictions can be visualized, but a single tree may be unstable: small changes in training data can produce a different structure.

Algorithm

1. Load the classification dataset and separate features X and target y.
2. Split the data into training and testing sets, preferably with stratification.
3. Choose a set of maximum depths, such as 1, 2, 3, 5, 8, and None.
4. For each depth, create a DecisionTreeClassifier with a fixed random_state and selected criterion.
5. Fit the classifier on the training data and predict the test labels.
6. Calculate training and test accuracy, and optionally precision, recall, and F1-score.
7. Plot depth against training and test accuracy.
8. Compare the results to identify underfitting, overfitting, and a reasonable depth; validate the final choice without tuning on the test set.

Experiment 10

Ensemble Learning Using Random Forest

Aim

To implement a Random Forest classifier and compare its performance with a single Decision Tree classifier.

Objectives

1. Understand ensemble learning and the bagging approach.
2. Train a Random Forest using multiple decision trees.
3. Compare Random Forest and Decision Tree performance on the same split.
4. Examine feature importance and generalization.

Theory

Ensemble learning combines predictions from multiple models to improve robustness or predictive performance. Random Forest is a bagging-based ensemble of decision trees. Each tree is trained on a bootstrap sample of the

training data, and each split considers a random subset of features.

For classification, the forest combines tree predictions by majority vote (or averages class probabilities). Random feature selection reduces correlation among trees, while aggregation often reduces variance compared with a single deep decision tree.

Important hyperparameters include the number of trees (n_estimators), maximum depth, minimum samples per split or leaf, and number of features considered at each split (max_features). Feature importance can provide a useful indication of predictive contribution, but it should not be interpreted as proof of causality.

Algorithm

1. Load the classification dataset and separate X and y.
2. Split the dataset into training and testing sets using a fixed random_state and suitable stratification.
3. Create a DecisionTreeClassifier and a RandomForestClassifier with documented settings.
4. Fit both models using the same training data.
5. Predict labels for the same test data using both models.
6. Calculate accuracy, precision, recall, and F1-score; compare confusion matrices if useful.
7. Display or plot Random Forest feature importances, noting their limitations.
8. Compare generalization, complexity, and results, and state which model performs better on the chosen evaluation data.

Experiment 11

K-Means Clustering and Elbow Method

Aim

To implement K-Means clustering on an unlabelled dataset and estimate the optimal number of clusters using the Elbow method.

Objectives

1. Understand unsupervised clustering and centroid-based grouping.
2. Apply K-Means to numerical data.
3. Use within-cluster sum of squares (inertia) to study cluster compactness.
4. Estimate a suitable cluster count using the Elbow method.

Theory

K-Means is an unsupervised algorithm that partitions observations into K clusters. Each cluster is represented by its centroid, the mean of the points assigned to that cluster. The algorithm alternates between assigning each point to its nearest centroid and updating centroids to the mean of assigned points.

The objective is to minimize the within-cluster sum of squared distances, also called inertia. The algorithm can converge to a local optimum, so results may depend on initialization. K-Means is sensitive to feature scale, outliers, and the chosen K, and works best when clusters are reasonably compact and roughly spherical.

The Elbow method fits K-Means for a range of K values and plots inertia against K. Inertia always decreases as K increases. A point where the decrease begins to slow (the elbow) is a possible useful K, but the elbow may not be clear and should be supported by domain knowledge or other cluster-validity measures.

Algorithm

1. Load an unlabelled dataset and select relevant numerical features.

2. Handle missing values and standardize or normalize features where scales differ.
3. Choose a range of candidate cluster counts, for example K = 1 to 10.
4. For each K, fit KMeans with a fixed random_state and n_init setting; record inertia.
5. Plot K against inertia and inspect the curve for an elbow.
6. Choose a reasonable K based on the curve and the problem context.
7. Fit the final K-Means model using the selected K and obtain cluster labels and centroids.
8. Visualize clusters using two features or a dimensionality-reduction projection, and describe the limitations.

Experiment 12

Hierarchical Agglomerative Clustering and Dendrogram

Aim

To implement agglomerative hierarchical clustering and visualize cluster relationships using a dendrogram.

Objectives

1. Understand bottom-up hierarchical clustering.
2. Study distance metrics and linkage criteria.
3. Construct and interpret a dendrogram.
4. Obtain cluster labels by selecting a suitable cut level.

Theory

Hierarchical clustering builds a nested hierarchy of clusters. In the agglomerative approach, every observation begins as an individual cluster. At each step, the closest pair of clusters is merged until a stopping condition is reached or all observations form one cluster.

Linkage defines the distance between clusters. Single linkage uses the minimum pairwise distance; complete linkage uses the maximum; average linkage uses the average pairwise distance; Ward linkage merges clusters to minimize the increase in within-cluster variance. The selected metric and linkage can strongly affect the result.

A dendrogram is a tree diagram showing the sequence and distance of merges. The vertical height of a merge represents linkage distance. Drawing a horizontal cut through the dendrogram gives a set of clusters. Hierarchical methods can be computationally demanding for large datasets, so a representative sample may be needed for visualization.

Algorithm

1. Load the unlabelled dataset, select relevant numerical features, and handle missing values.
2. Scale the features when distance calculations would otherwise be dominated by variables with large ranges.
3. Choose a distance metric and linkage method appropriate to the data.
4. Compute the hierarchical linkage matrix using scipy.cluster.hierarchy.linkage.
5. Plot a dendrogram and inspect large vertical gaps between merges.
6. Choose a cut height or desired number of clusters based on the dendrogram and context.
7. Fit AgglomerativeClustering with the selected number of clusters and compatible linkage/metric settings.
8. Assign cluster labels, visualize the groups where possible, and interpret the cluster structure.

Experiment 13

DBSCAN Clustering and Outlier Detection

Aim

To implement DBSCAN clustering to identify dense groups of observations and detect noise or outlier points.

Objectives

1. Understand density-based clustering.
2. Study the roles of eps and min_samples.
3. Identify core, border, and noise points.
4. Visualize clusters and examine observations labelled as noise.

Theory

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) groups observations that form sufficiently dense regions. It uses two main parameters: eps, the neighbourhood radius, and min_samples, the minimum number of observations required in an eps-neighbourhood for a point to be a core point.

A core point has at least min_samples points in its neighbourhood (including itself under the common convention). A border point lies within the neighbourhood of a core point but does not itself meet the core-point density requirement. A noise point is not assigned to any cluster and is commonly labelled -1.

DBSCAN can discover irregularly shaped clusters and explicitly identify noise, unlike K-Means. Its performance depends strongly on feature scaling and parameter choice. A single eps may not work well when clusters have very different densities or when data dimensionality is high.

Algorithm

1. Load the unlabelled dataset and select relevant numerical features.
2. Handle missing values and scale features so distance calculations are meaningful.
3. Select candidate eps and min_samples values; a k-distance plot can help estimate eps.
4. Create a DBSCAN model with the selected parameters and fit it to the scaled data.
5. Obtain the cluster labels; identify points labelled -1 as noise.
6. Count the number of clusters excluding the noise label and count noise observations.
7. Visualize the clusters and noise points using two features or a suitable projection.
8. Experiment with parameter values and explain how cluster count and noise assignments change.

Experiment 14

Principal Component Analysis (PCA) for Dimensionality Reduction

Aim

To apply Principal Component Analysis for dimensionality reduction and analyze how much data variance is preserved.

Objectives

1. Understand the purpose of dimensionality reduction.
2. Standardize features and compute principal components.
3. Transform the original data into a lower-dimensional representation.

4. Analyze explained variance and cumulative variance preservation.

Theory

Principal Component Analysis is an unsupervised linear transformation that converts possibly correlated features into a set of orthogonal components. The first principal component captures the greatest possible variance; each subsequent component captures the greatest remaining variance subject to being orthogonal to previous components.

PCA is commonly computed using an eigen-decomposition of the covariance matrix or singular value decomposition. Since variance depends on feature scale, standardization is usually appropriate when features use different units or ranges. PCA is sensitive to outliers and captures linear structure.

Explained variance ratio indicates the fraction of total variance represented by each component. The cumulative explained variance ratio for the first m components indicates the fraction retained after reducing to m dimensions. A threshold such as 90% or 95% is sometimes used, but the appropriate threshold depends on the task.

PCA components are combinations of original features, so transformed features may be less interpretable. PCA should be fitted on training data only when used in a predictive workflow, then applied to validation and test data.

Algorithm

1. Load the dataset and select numerical features; handle missing values.
2. Split into training and testing data if PCA is part of a predictive workflow.
3. Standardize the features by fitting the scaler on training data only.
4. Create a PCA estimator and fit it to the scaled training features.
5. Transform the data into principal-component coordinates.
6. Display each component's explained_variance_ratio_ and calculate the cumulative sum.
7. Plot individual and cumulative explained variance against component number.
8. Select the number of components based on a justified variance threshold or downstream validation performance.
9. Report the reduced dimensions and percentage of variance preserved; optionally compare downstream model performance before and after PCA.

Experiment 15

Grid Search and Cross-Validation for Hyperparameter Tuning

Aim

To improve machine learning model performance by tuning hyperparameters using Grid Search and Cross-Validation.

Objectives

1. Distinguish model parameters learned during fitting from user-selected hyperparameters.
2. Understand k-fold cross-validation and grid search.
3. Define a parameter grid and evaluate candidate combinations systematically.
4. Select the best configuration using training-only validation and assess it on a held-out test set.

Theory

Hyperparameters control the learning process or model complexity and are set before model fitting. Examples include K in KNN, maximum tree depth, regularization strength, and the number of trees in a Random Forest. Hyperparameter tuning searches for settings that generalize well.

In k-fold cross-validation, the training data is divided into k folds. The model is trained on k-1 folds and validated on the remaining fold; this is repeated so each fold serves as validation once. The scores are averaged to estimate performance more reliably than a single split, although folds are not fully independent.

GridSearchCV evaluates all specified combinations in a parameter grid using cross-validation and selects the combination with the best mean validation score. The scoring metric should match the task and the cost of different errors. A Pipeline allows preprocessing to be refitted within each fold and prevents leakage.

The test set must remain untouched during grid search and model selection. After selecting the best estimator using training data, evaluate it once on the held-out test set to obtain an unbiased final estimate. Large grids and many folds increase computation time.

Algorithm

1. Load the dataset, define X and y, and create a held-out train-test split.
2. Build a Pipeline containing all required preprocessing steps and the estimator.
3. Choose an estimator and define a parameter grid containing candidate hyperparameter values.
4. Select an appropriate cross-validation strategy, such as StratifiedKFold for classification or KFold for regression.
5. Create GridSearchCV with the Pipeline, parameter grid, cross-validation strategy, scoring metric, and suitable n_jobs setting.
6. Fit GridSearchCV using only the training data.
7. Display best_params_, best_score_, and the cross-validation results.
8. Use best_estimator_ to predict the held-out test data.
9. Evaluate the final model using suitable test metrics and compare with a reasonable baseline model.
10. Report the chosen hyperparameters, validation method, final test results, and any computational limitations.

Suggested Practical Record Completion

For each experiment, add the following items if they are required by your college format:

- Dataset name, source, number of rows and columns, and target/features used.
- Python code written and executed in Jupyter Notebook.
- Important output, graphs, tables, and evaluation metric values.
- A brief result/conclusion stating what was implemented and what was observed.
- Any limitations, assumptions, or parameter choices relevant to the experiment.